



OPERATING SYSTEM

OPEN ENDED LAB



Submitted To	Mam Aqsa
Submitted By	Abdul Mohiz
CMIS ID	534658
Course	BESE 30-B
Date	04 April 2026

1. Objective

To implement and compare all four classical Dynamic Memory Allocation algorithms within a single OS-level simulation, demonstrating how each algorithm manages a shared memory pool differently across allocation, deallocation, block splitting, block merging (coalescing), and a randomised stress test.

2. Memory Layout

The memory pool totals 1,024 KB divided into 10 initially free blocks of varying sizes. The same configuration is used as an independent copy for each of the four algorithms, ensuring a fair comparison.

Blk ID	Start(KB)	End(KB)	Size(KB)	Status	PID
0	0	80	80	FREE	---
1	80	280	200	FREE	---
2	280	330	50	FREE	---
3	330	430	100	FREE	---
4	430	460	30	FREE	---
5	460	580	120	FREE	---
6	580	620	40	FREE	---
7	620	770	150	FREE	---
8	770	830	60	FREE	---
9	830	1024	194	FREE	---

ASCII Memory Diagram:

```
Addr:  0      80     280   330   430   460   580   620   770   830     1024
      |--80--|--200--|--50--|--100--|--30--|--120--|--40--|--150--|--60--|--194--|
Block: [Blk0] [Blk1] [Blk2] [Blk3] [Blk4] [Blk5] [Blk6] [Blk7] [Blk8] [Blk9]
Status:[FREE] [FREE] [FREE] [FREE] [FREE] [FREE] [FREE] [FREE] [FREE] [FREE]
```

3. Algorithm Descriptions

3.1 First-Fit

First-Fit scans the block list from the beginning and allocates the first free block large enough to satisfy the request. It is the simplest and fastest strategy because the search terminates as soon as a suitable block is found, giving it an effective $O(1)$ to $O(n)$ search time depending on block position.

- Stops at the very first suitable block — no need to scan the entire list.
- Tends to leave small holes near the beginning of memory over time.
- External fragmentation builds up at the front; later requests must scan further.

3.2 Next-Fit

Next-Fit is a variation of First-Fit that remembers where the last allocation was made and resumes scanning from that position for the next request. This distributes allocations more evenly across memory.

- Avoids repeatedly splitting the same blocks at the start of the list.
- Can wrap around to the beginning if no suitable block is found before the end.
- Tends to create more fragmentation uniformly across the entire memory space.

3.3 Best-Fit

Best-Fit searches the entire free-block list every time and picks the smallest block that is still large enough for the request. This minimises wasted space inside the allocated block.

- Preserves large blocks for future big requests.
- Slower because it must scan every free block on every allocation request.
- Creates many tiny leftover fragments over time, increasing long-run external fragmentation.

3.4 Worst-Fit

Worst-Fit always selects the largest available free block. The rationale is that breaking off a piece from the largest block leaves a bigger remainder, which is more likely to satisfy a future medium-sized request.

- Produces larger remnant blocks compared to Best-Fit.
- Exhausts large blocks quickly, which can prevent future big allocations from succeeding.
- Initially reduces small-hole fragmentation; degrades as large blocks are consumed.

4. Simulation Results

Five allocation requests and two deallocation requests were issued against each algorithm's independent memory pool copy. Request #5 (600 KB) was designed to fail since no single contiguous block of 600 KB exists in the initial layout.

4.1 Allocation Request Results

Req#	PID	Size (KB)	First-Fit	Next-Fit	Best-Fit	Worst-Fit
1	101	80	Blk 0 (exact)	Blk 0 (exact)	Blk 0 (exact)	Blk 1 (split->Blk 10)
2	102	200	Blk 1 (exact)	Blk 1 (exact)	Blk 1 (exact)	FAIL (no 200KB block)
3	103	45	Blk 2 (exact)	Blk 2 (exact)	Blk 2 (exact)	Blk 9 (split)
4	104	130	Blk 7 (exact)	Blk 7 (exact)	Blk 7 (exact)	Blk 7 (exact)
5	105	600	FAIL	FAIL	FAIL	FAIL

Request Analysis: Requests 1–4 succeeded for First-Fit, Next-Fit, and Best-Fit. Worst-Fit behaved differently for request #1 — it chose Block 1 (200 KB, the largest block) instead of Block 0 (80 KB exact fit), splitting it and immediately making a 200 KB request impossible. This demonstrates Worst-Fit's key weakness: consuming large blocks prematurely.

4.2 Deallocation and Merging

After deallocating PID 101 and PID 103, the coalescing routine merged several adjacent free blocks:

- Deallocating PID 101 (Block 0) and PID 103 (Block 2) in First-Fit, Next-Fit, and Best-Fit triggered merges of Blocks 3+4+5+6 into 290 KB and Blocks 8+9 into 254 KB, then a further merge of Blocks 2+3 into 340 KB.
- In Worst-Fit, the earlier split of Block 1 resulted in a massive chain merge after PID 101 was freed, coalescing Blocks 0+1+10+2+3+4+5+6 into a single 620 KB contiguous region.
- Every merge event was printed to output with block IDs and resulting sizes.

5. Fragmentation Analysis

5.1 Internal Fragmentation

Internal fragmentation occurs when the allocated block is larger than what the process needs and the leftover is too small to split (i.e., ≤ 20 KB).

- First-Fit: Moderate. Several blocks had small leftovers absorbed without splitting (e.g., Block 0 allocated exactly 80 KB — no waste; Block 7 allocated 150 KB for a 130 KB request — 20 KB absorbed).
- Next-Fit: Moderate. Same pattern as First-Fit for this workload since both traversed the list from a similar position.
- Best-Fit: Minimal. Best-Fit sought the closest-fitting block, minimising absorbed waste.
- Worst-Fit: Low-to-moderate initially, but caused a failure for PID 102 (200 KB) because the 200 KB block had already been split.

5.2 External Fragmentation

External fragmentation refers to free memory that cannot satisfy a request because it is scattered across multiple non-contiguous blocks.

- All four algorithms showed external fragmentation after the main simulation due to alternating allocated and free blocks.
- After the stress test, First-Fit left 2 scattered free blocks (81 KB total), Best-Fit left 1 free block (64 KB), Next-Fit left 0 free blocks (fully utilised), and Worst-Fit left 2 free blocks (216 KB total).
- Coalescing significantly reduced external fragmentation after each deallocation, demonstrating the importance of merging.

5.3 Post-Stress Fragmentation Report

Metric	First-Fit	Next-Fit	Best-Fit	Worst-Fit	Notes
Requests	10	10	10	10	Fixed random seed
Succeeded	9	9	9	10	Worst-Fit allocated all 10
Failed	1	1	1	0	
Free After (KB)	81	0	64	216	Worst-Fit left most free
Free Blocks	2	0	1	2	
Ext. Frag?	YES	NO	NO	YES	NF fully utilised pool

6. Stress Test

After the main simulation, 10 random allocation requests (sizes 10–150 KB, fixed seed 42 for reproducibility) were issued to each algorithm's pool. The same request sequence was used for all four algorithms.

6.1 Stress Test Behaviour

- First-Fit allocated 9/10 requests. It consistently scanned from the front, which rapidly exhausted the 80 KB Block 0 first. One 73 KB request failed because the only remaining free blocks were 44 KB and 37 KB.
- Next-Fit allocated 9/10 requests. Starting from the cursor left by the last main-simulation allocation, it spread stress requests across Block 8 and Block 2 before wrapping around. It fully consumed the pool with 0 KB free remaining.
- Best-Fit allocated 9/10 requests. It tightly matched each request to the smallest fitting block. One 73 KB request failed (only 64 KB block remained). It produced the fewest wasted bytes of internal fragmentation.
- Worst-Fit allocated all 10/10 requests. By always taking from the largest block (620 KB free block after coalescing), it kept producing large-enough remnants for every subsequent request. This highlights Worst-Fit's advantage when a single very large block dominates the pool.

7. Comparative Justification

Algorithm	Alloc Speed	Int. Frag	Ext. Frag	Best Use
First-Fit	Fast $O(n)$	Moderate	Moderate	General-purpose OS
Next-Fit	Fastest $O(1)$	Moderate	High	High-throughput systems
Best-Fit	Slow $O(n)$	Minimal	High (long-run)	Memory-constrained env
Worst-Fit	Slow $O(n)$	Low-Mod	Low (short-run)	Varied process sizes

7.1 Allocation Speed

Next-Fit is the fastest in practice because its cursor avoids repeatedly scanning already-allocated blocks at the start. First-Fit is nearly as fast for early allocations but degrades as the beginning of the list fills up. Best-Fit and Worst-Fit both require a full scan of all free blocks on every allocation, making them $O(n)$ with larger constants in practice.

7.2 Internal Fragmentation

Best-Fit produces the least internal fragmentation because it seeks the block closest in size to the request. Worst-Fit also performs reasonably in this regard because it always leaves a large remainder. First-Fit and Next-Fit produce moderate internal fragmentation since they stop at the first or next suitable block regardless of fit quality.

7.3 External Fragmentation

All algorithms suffer from external fragmentation over time, but the severity and timing differ. Best-Fit creates many small holes as it carves tight slices, leading to high long-run external fragmentation. Worst-Fit initially leaves large remnants (low external fragmentation) but eventually exhausts large blocks, after which performance collapses. First-Fit concentrates holes near the start; Next-Fit spreads them. Coalescing (merging) partially mitigates external fragmentation for all algorithms.

7.4 Suitability

- First-Fit: Best for general-purpose operating systems where simplicity and speed matter more than optimal memory usage.
- Next-Fit: Best for high-throughput environments where allocation requests arrive rapidly and even distribution of holes is preferable to concentrated fragmentation.
- Best-Fit: Best for memory-constrained embedded systems where every kilobyte counts and slower allocation is acceptable.
- Worst-Fit: Best for workloads with highly variable process sizes where leaving large usable remnants is more important than speed or long-term hole management.

8. Source Code:

GNU nano 6.2

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>
#include <climits>
#include <iomanip>
#include <algorithm>

using namespace std;

struct Block {
    int id;
    int startAddress;
    int size;
    bool isFree;
    int processId;

    Block(int id, int start, int size, bool free = true, int pid = -1)
        : id(id), startAddress(start), size(size), isFree(free), processId(pid) {}
};

struct MemoryPool {
    vector<Block> blocks;
    string algorithmName;
    int nextFitIndex;

    MemoryPool(const string& name, const vector<Block>& initialBlocks)
        : algorithmName(name), blocks(initialBlocks), nextFitIndex(0) {}
};

void printSeparator(char c = '-', int w = 90) {
    cout << string(w, c) << "\n";
}

void displayMemory(MemoryPool& pool) {
    cout << "\n [" << pool.algorithmName << "] Memory State:\n";
    printSeparator();
    cout << left
        << setw(8) << "BlkID"
        << setw(12) << "Start(KB)"
        << setw(12) << "Size(KB)"
        << setw(12) << "Status"
        << setw(10) << "PID"
        << "\n";
    printSeparator();
    for (auto& b : pool.blocks) {
        cout << left
            << setw(8) << b.id
            << setw(12) << b.startAddress
            << setw(12) << b.size
            << setw(12) << (b.isFree ? "FREE" : "ALLOCATED")
            << setw(10) << (b.isFree ? "----" : to_string(b.processId))
            << "\n";
    }
}
```



```

    }
    printSeparator();
}

void printFragmentationReport(MemoryPool& pool) {
    int totalFree = 0, freeBlocks = 0, largestFree = 0, totalAllocated = 0;
    for (auto& b : pool.blocks) {
        if (b.isFree) {
            totalFree += b.size;
            freeBlocks++;
            largestFree = max(largestFree, b.size);
        } else {
            totalAllocated += b.size;
        }
    }
    bool extFrag = (freeBlocks > 1);
    cout << "\n --- Fragmentation Report [" << pool.algorithmName << "] ---\n";
    cout << "  Total Free Memory   : " << totalFree << " KB\n";
    cout << "  Total Allocated       : " << totalAllocated << " KB\n";
    cout << "  Free Block Count      : " << freeBlocks << "\n";
    cout << "  Largest Free Block    : " << largestFree << " KB\n";
    cout << "  External Frag         : " << (extFrag ? "YES (scattered free blocks)" : "NO") << "\n";
    printSeparator();
}

void splitBlock(MemoryPool& pool, int idx, int requestedSize, int pid) {
    Block& b = pool.blocks[idx];
    int leftover = b.size - requestedSize;

    b.isFree = false;
    b.processId = pid;
    int origSize = b.size;
    b.size = requestedSize;

    if (leftover > 20) {
        int newId = pool.blocks.back().id + 1;
        int newStart = b.startAddress + requestedSize;
        Block newBlock(newId, newStart, leftover, true, -1);
        pool.blocks.insert(pool.blocks.begin() + idx + 1, newBlock);
        cout << "  [SPLIT] Block " << b.id
              << " split: " << requestedSize << " KB allocated, "
              << leftover << " KB returned as Block " << newId << "\n";
    } else {
        b.size = origSize;
        cout << "  [ALLOC] Block " << b.id
              << " allocated (" << origSize << " KB, no split - leftover <= 20 KB)\n";
    }
}

void mergeBlocks(MemoryPool& pool) {
    bool merged = true;
    while (merged) {
        merged = false;
        for (int i = 0; i + 1 < (int)pool.blocks.size(); i++) {

```

```

    }
    printSeparator();
}

void printFragmentationReport(MemoryPool& pool) {
    int totalFree = 0, freeBlocks = 0, largestFree = 0, totalAllocated = 0;
    for (auto& b : pool.blocks) {
        if (b.isFree) {
            totalFree += b.size;
            freeBlocks++;
            largestFree = max(largestFree, b.size);
        } else {
            totalAllocated += b.size;
        }
    }
    bool extFrag = (freeBlocks > 1);
    cout << "\n --- Fragmentation Report [" << pool.algorithmName << "] ---\n";
    cout << "  Total Free Memory   : " << totalFree << " KB\n";
    cout << "  Total Allocated       : " << totalAllocated << " KB\n";
    cout << "  Free Block Count      : " << freeBlocks << "\n";
    cout << "  Largest Free Block    : " << largestFree << " KB\n";
    cout << "  External Frag         : " << (extFrag ? "YES (scattered free blocks)" : "NO") << "\n";
    printSeparator();
}

void splitBlock(MemoryPool& pool, int idx, int requestedSize, int pid) {
    Block& b = pool.blocks[idx];
    int leftover = b.size - requestedSize;

    b.isFree = false;
    b.processId = pid;
    int origSize = b.size;
    b.size = requestedSize;

    if (leftover > 20) {
        int newId = pool.blocks.back().id + 1;
        int newStart = b.startAddress + requestedSize;
        Block newBlock(newId, newStart, leftover, true, -1);
        pool.blocks.insert(pool.blocks.begin() + idx + 1, newBlock);
        cout << "  [SPLIT] Block " << b.id
              << " split: " << requestedSize << " KB allocated, "
              << leftover << " KB returned as Block " << newId << "\n";
    } else {
        b.size = origSize;
        cout << "  [ALLOC] Block " << b.id
              << " allocated (" << origSize << " KB, no split - leftover <= 20 KB)\n";
    }
}

void mergeBlocks(MemoryPool& pool) {
    bool merged = true;
    while (merged) {
        merged = false;
        for (int i = 0; i + 1 < (int)pool.blocks.size(); i++) {

```

```

        if (diff < bestDiff) {
            bestDiff = diff;
            bestIdx = i;
        }
    }
}
if (bestIdx == -1) {
    cout << " [FAIL] Best-Fit: No block large enough for " << size << " KB (PID " << pid << ")\n";
    return false;
}
splitBlock(pool, bestIdx, size, pid);
return true;
}

bool worstFit(MemoryPool& pool, int size, int pid) {
    int worstIdx = -1;
    int worstSize = -1;
    for (int i = 0; i < (int)pool.blocks.size(); i++) {
        if (pool.blocks[i].isFree && pool.blocks[i].size >= size) {
            if (pool.blocks[i].size > worstSize) {
                worstSize = pool.blocks[i].size;
                worstIdx = i;
            }
        }
    }
    if (worstIdx == -1) {
        cout << " [FAIL] Worst-Fit: No block large enough for " << size << " KB (PID " << pid << ")\n";
        return false;
    }
    splitBlock(pool, worstIdx, size, pid);
    return true;
}

void stressTest(MemoryPool& pool,
                bool (*allocFn)(MemoryPool&, int, int)) {
    cout << "\n ===== STRESS TEST [" << pool.algorithmName << "] =====\n";
    srand(42);
    int success = 0, fail = 0;
    for (int i = 0; i < 10; i++) {
        int sz = 10 + rand() % 141;
        int pid = 900 + i;
        cout << " Stress req #" << (i+1) << ": " << sz << " KB (PID " << pid << ") -> ";
        if (allocFn(pool, sz, pid)) success++;
        else fail++;
    }
    cout << " Result: " << success << " allocated, " << fail << " failed\n";
    printFragmentationReport(pool);
}

void runSimulation(MemoryPool& pool,
                  bool (*allocFn)(MemoryPool&, int, int),
                  void (*stressFn)(MemoryPool&, bool (*)(MemoryPool&,int,int))) {
    cout << "\n";

```

```

int main() {
    auto makePool = [](const string& name) {
        vector<Block> blocks = {
            Block(0,0,80),
            Block(1,80,200),
            Block(2,280,50),
            Block(3,330,100),
            Block(4,430,30),
            Block(5,460,120),
            Block(6,580,40),
            Block(7,620,150),
            Block(8,770,60),
            Block(9,830,194)
        };
        return MemoryPool(name, blocks);
    };

    MemoryPool ffPool = makePool("First-Fit");
    MemoryPool nfPool = makePool("Next-Fit");
    MemoryPool bfPool = makePool("Best-Fit");
    MemoryPool wfPool = makePool("Worst-Fit");

    runSimulation(ffPool, firstFit, stressTest);
    runSimulation(nfPool, nextFit, stressTest);
    runSimulation(bfPool, bestFit, stressTest);
    runSimulation(wfPool, worstFit, stressTest);

    cout << "\n";
    printSeparator('=');
    cout << "  COMPARATIVE SUMMARY TABLE\n";
    printSeparator('=');

    cout << left
        << setw(14) << "Algorithm"
        << setw(18) << "Alloc Speed"
        << setw(20) << "Internal Frag"
        << setw(20) << "External Frag"
        << setw(22) << "Best Use Case"
        << "\n";

    printSeparator();

    cout << left
        << setw(14) << "First-Fit"

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify


```

printSeparator();

cout << left
    << setw(14) << "First-Fit"
    << setw(18) << "Fast (O(n))"
    << setw(20) << "Moderate"
    << setw(20) << "Moderate"
    << setw(22) << "General purpose"
    << "\n";

cout << left
    << setw(14) << "Next-Fit"
    << setw(18) << "Fastest"
    << setw(20) << "Moderate-High"
    << setw(20) << "High"
    << setw(22) << "High-throughput sys"
    << "\n";

cout << left
    << setw(14) << "Best-Fit"
    << setw(18) << "Slow (O(n))"
    << setw(20) << "Minimal"
    << setw(20) << "High (long-term)"
    << setw(22) << "Memory-constrained"
    << "\n";

cout << left
    << setw(14) << "Worst-Fit"
    << setw(18) << "Slow (O(n))"
    << setw(20) << "Low-Moderate"
    << setw(20) << "Low (initially)"
    << setw(22) << "Varied process sizes"
    << "\n";

printSeparator('=');

cout << "\n Simulation complete.\n\n";
return 0;
}

```

9.Outputs:

1. First-Fit:

```
root@DESKTOP-09TR4UUF:~# cd oel
root@DESKTOP-09TR4UUF:~/oel# g++ mohiz2.cpp -o memory
root@DESKTOP-09TR4UUF:~/oel# ./memory
```

ALGORITHM: First-Fit

[First-Fit] Memory State:

BlkID	Start(KB)	Size(KB)	Status	PID
0	0	80	FREE	---
1	80	200	FREE	---
2	280	50	FREE	---
3	330	100	FREE	---
4	430	30	FREE	---
5	460	120	FREE	---
6	580	40	FREE	---
7	620	150	FREE	---
8	770	60	FREE	---
9	830	194	FREE	---

>> Allocate 80 KB to PID 101

[ALLOC] Block 0 allocated (80 KB, no split - leftover <= 20 KB)

[First-Fit] Memory State:

BlkID	Start(KB)	Size(KB)	Status	PID
0	0	80	ALLOCATED	101
1	80	200	FREE	---
2	280	50	FREE	---
3	330	100	FREE	---
4	430	30	FREE	---
5	460	120	FREE	---
6	580	40	FREE	---
7	620	150	FREE	---
8	770	60	FREE	---
9	830	194	FREE	---

>> Allocate 200 KB to PID 102

[ALLOC] Block 1 allocated (200 KB, no split - leftover <= 20 KB)

[First-Fit] Memory State:

BlkID	Start(KB)	Size(KB)	Status	PID
0	0	80	ALLOCATED	101
1	80	200	ALLOCATED	102
2	280	50	FREE	---
3	330	100	FREE	---
4	430	30	FREE	---
5	460	120	FREE	---
6	580	40	FREE	---
7	620	150	FREE	---

>> Deallocate PID 101

[FREE] Process 101 block 0 (80 KB) deallocated
[MERGE] Blocks 3 and 4 merged into one 130 KB block
[MERGE] Blocks 3 and 5 merged into one 250 KB block
[MERGE] Blocks 3 and 6 merged into one 290 KB block
[MERGE] Blocks 8 and 9 merged into one 254 KB block

[First-Fit] Memory State:

BlkID	Start(KB)	Size(KB)	Status	PID
0	0	80	FREE	---
1	80	200	ALLOCATED	102
2	280	50	ALLOCATED	103
3	330	290	FREE	---
7	620	150	ALLOCATED	104
8	770	254	FREE	---

>> Deallocate PID 103

[FREE] Process 103 block 2 (50 KB) deallocated
[MERGE] Blocks 2 and 3 merged into one 340 KB block

[First-Fit] Memory State:

BlkID	Start(KB)	Size(KB)	Status	PID
0	0	80	FREE	---
1	80	200	ALLOCATED	102
2	280	340	FREE	---
7	620	150	ALLOCATED	104
8	770	254	FREE	---

--- Fragmentation Report [First-Fit] ---

Total Free Memory : 674 KB
Total Allocated : 350 KB
Free Block Count : 3
Largest Free Block : 340 KB
External Frag : YES (scattered free blocks)

===== STRESS TEST [First-Fit] =====

Stress req #1: 16 KB (PID 900) -> [SPLIT] Block 0 split: 16 KB allocated, 64 KB returned as Block 9
Stress req #2: 70 KB (PID 901) -> [SPLIT] Block 2 split: 70 KB allocated, 270 KB returned as Block 9
Stress req #3: 87 KB (PID 902) -> [SPLIT] Block 9 split: 87 KB allocated, 183 KB returned as Block 9
Stress req #4: 139 KB (PID 903) -> [SPLIT] Block 9 split: 139 KB allocated, 44 KB returned as Block 9
Stress req #5: 86 KB (PID 904) -> [SPLIT] Block 8 split: 86 KB allocated, 168 KB returned as Block 9
Stress req #6: 77 KB (PID 905) -> [SPLIT] Block 9 split: 77 KB allocated, 91 KB returned as Block 10
Stress req #7: 83 KB (PID 906) -> [ALLOC] Block 10 allocated (91 KB, no split - leftover <= 20 KB)
Stress req #8: 73 KB (PID 907) -> [FAIL] First-Fit: No block large enough for 73 KB (PID 907)
Stress req #9: 15 KB (PID 908) -> [SPLIT] Block 9 split: 15 KB allocated, 49 KB returned as Block 11
Stress req #10: 12 KB (PID 909) -> [SPLIT] Block 11 split: 12 KB allocated, 37 KB returned as Block 11
Result: 9 allocated, 1 failed

--- Fragmentation Report [First-Fit] ---

2.Next-Fit:

```
--- Fragmentation Report [First-Fit] ---
Total Free Memory : 81 KB
Total Allocated : 943 KB
Free Block Count : 2
Largest Free Block : 444 KB
External Frag : YES (scattered free blocks)

=====

ALGORITHM: Next-Fit

=====

[Next-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      FREE    ---
80     80          200     FREE    ---
280    80           50      FREE    ---
330    100         100     FREE    ---
430    30           30      FREE    ---
460    120         120     FREE    ---
580    40           40      FREE    ---
620    150         150     FREE    ---
770    60           60      FREE    ---
830    194          194     FREE    ---

=====

>> Allocate 80 KB to PID 101
[ALLOC] Block 0 allocated (80 KB, no split - leftover <= 20 KB)

[Next-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      ALLOCATED 101
80     80          200     FREE    ---
280    80           50      FREE    ---
330    100         100     FREE    ---
430    30           30      FREE    ---
460    120         120     FREE    ---
580    40           40      FREE    ---
620    150         150     FREE    ---
770    60           60      FREE    ---
830    194          194     FREE    ---

=====

>> Allocate 200 KB to PID 102
[ALLOC] Block 1 allocated (200 KB, no split - leftover <= 20 KB)

[Next-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      ALLOCATED 101
80     80          200     ALLOCATED 102
280    50           50      FREE    ---
330    100         100     FREE    ---
```

```
6      580         40      FREE    ---
7      620         150     ALLOCATED 104
8      770          60      FREE    ---
9      830         194     FREE    ---

=====

>> Deallocate PID 101
[FREE] Process 101 block 0 (80 KB) deallocated
[MERGE] Blocks 3 and 4 merged into one 130 KB block
[MERGE] Blocks 3 and 5 merged into one 250 KB block
[MERGE] Blocks 3 and 6 merged into one 290 KB block
[MERGE] Blocks 8 and 9 merged into one 254 KB block

[Next-Fit] Memory State:
```

```
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      FREE    ---
1      80          200     ALLOCATED 102
2      280         50      ALLOCATED 103
3      330         290     FREE    ---
7      620         150     ALLOCATED 104
8      770         254     FREE    ---

=====
```

```
>> Deallocate PID 103
[FREE] Process 103 block 2 (50 KB) deallocated
[MERGE] Blocks 2 and 3 merged into one 340 KB block

[Next-Fit] Memory State:
```

```
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      FREE    ---
1      80          200     ALLOCATED 102
2      280         340     FREE    ---
7      620         150     ALLOCATED 104
8      770         254     FREE    ---

=====
```

```
--- Fragmentation Report [Next-Fit] ---
Total Free Memory : 674 KB
Total Allocated : 350 KB
Free Block Count : 3
Largest Free Block : 340 KB
External Frag : YES (scattered free blocks)

=====
```

```
===== STRESS TEST [Next-Fit] =====
Stress req #1: 16 KB (PID 980) -> [SPLIT] Block 8 split: 16 KB allocated, 238 KB returned as Block 9
Stress req #2: 70 KB (PID 981) -> [ALLOC] Block 0 allocated (80 KB, no split - leftover <= 20 KB)
Stress req #3: 87 KB (PID 982) -> [SPLIT] Block 2 split: 87 KB allocated, 253 KB returned as Block 10
Stress req #4: 139 KB (PID 983) -> [SPLIT] Block 10 split: 139 KB allocated, 114 KB returned as Block 10
Stress req #5: 86 KB (PID 984) -> [SPLIT] Block 10 split: 86 KB allocated, 28 KB returned as Block 10
Stress req #6: 77 KB (PID 985) -> [SPLIT] Block 9 split: 77 KB allocated, 161 KB returned as Block 10
Stress req #7: 83 KB (PID 986) -> [SPLIT] Block 10 split: 83 KB allocated, 78 KB returned as Block 11
Stress req #8: 73 KB (PID 987) -> [ALLOC] Block 11 allocated (78 KB, no split - leftover <= 20 KB)
Stress req #9: 15 KB (PID 988) -> [ALLOC] Block 10 allocated (28 KB, no split - leftover <= 20 KB)
```

3.Best-fit:

```
9      830         194     FREE    ---

=====

>> Allocate 200 KB to PID 102
[ALLOC] Block 1 allocated (200 KB, no split - leftover <= 20 KB)

[Best-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      ALLOCATED 101
1      80          200     ALLOCATED 102
2      280         50      FREE    ---
3      330         100     FREE    ---
4      430         30      FREE    ---
5      460         120     FREE    ---
6      580         40      FREE    ---
7      620         150     FREE    ---
8      770         60      FREE    ---
9      830         194     FREE    ---

=====

>> Allocate 45 KB to PID 103
[ALLOC] Block 2 allocated (50 KB, no split - leftover <= 20 KB)

[Best-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      ALLOCATED 101
1      80          200     ALLOCATED 102
2      280         50      ALLOCATED 103
3      330         100     FREE    ---
4      430         30      FREE    ---
5      460         120     FREE    ---
6      580         40      FREE    ---
7      620         150     FREE    ---
8      770         60      FREE    ---
9      830         194     FREE    ---

=====

>> Allocate 130 KB to PID 104
[ALLOC] Block 7 allocated (150 KB, no split - leftover <= 20 KB)

[Best-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      ALLOCATED 101
1      80          200     ALLOCATED 102
2      280         50      ALLOCATED 103
3      330         100     FREE    ---
4      430         30      FREE    ---
5      460         120     FREE    ---
6      580         40      FREE    ---
7      620         150     ALLOCATED 104
8      770         60      FREE    ---
9      830         194     FREE    ---
```

```
>> Allocate 600 KB to PID 105
[FAIL] Best-Fit: No block large enough for 600 KB (PID 105)
```

```
[Best-Fit] Memory State:

BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      ALLOCATED 101
1      80          200     ALLOCATED 102
2      280         50      ALLOCATED 103
3      330         180     FREE    ---
4      430         30      FREE    ---
5      460         120     FREE    ---
6      580         40      FREE    ---
7      620         150     ALLOCATED 104
8      770         60      FREE    ---
9      830         194     FREE    ---

=====
```

```
>> Deallocate PID 101
[FREE] Process 101 block 0 (80 KB) deallocated
[MERGE] Blocks 3 and 4 merged into one 130 KB block
[MERGE] Blocks 3 and 5 merged into one 250 KB block
[MERGE] Blocks 3 and 6 merged into one 290 KB block
[MERGE] Blocks 8 and 9 merged into one 254 KB block

[Best-Fit] Memory State:
```

```
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      FREE    ---
1      80          200     ALLOCATED 102
2      280         50      ALLOCATED 103
3      330         290     FREE    ---
7      620         150     ALLOCATED 104
8      770         254     FREE    ---

=====
```

```
>> Deallocate PID 103
[FREE] Process 103 block 2 (50 KB) deallocated
[MERGE] Blocks 2 and 3 merged into one 340 KB block

[Best-Fit] Memory State:
```

```
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0           80      FREE    ---
1      80          200     ALLOCATED 102
2      280         340     FREE    ---
7      620         150     ALLOCATED 104
8      770         254     FREE    ---

=====
```

```
--- Fragmentation Report [Best-Fit] ---
Total Free Memory : 674 KB
Total Allocated : 350 KB
Free Block Count : 3
Largest Free Block : 340 KB
```

4. Worst-Fit:

```
[Worst-Fit] Memory State:
-----
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0         80      FREE    ---
1      80        200     FREE    ---
2      280       50      FREE    ---
3      330       100     FREE    ---
4      430       30      FREE    ---
5      460       120     FREE    ---
6      580       40      FREE    ---
7      620       150     FREE    ---
8      770       60      FREE    ---
9      830       194     FREE    ---
-----

>> Allocate 80 KB to PID 101
[SPLIT] Block 1 split: 80 KB allocated, 120 KB returned as Block 10

[Worst-Fit] Memory State:
-----
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0         80      FREE    ---
1      80        80      ALLOCATED 101
10     160       120     FREE    ---
2      280       50      FREE    ---
3      330       100     FREE    ---
4      430       30      FREE    ---
5      460       120     FREE    ---
6      580       40      FREE    ---
7      620       150     FREE    ---
8      770       60      FREE    ---
9      830       194     FREE    ---
-----

>> Allocate 200 KB to PID 102
[FAIL] Worst-Fit: No block large enough for 200 KB (PID 102)

[Worst-Fit] Memory State:
-----
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0         80      FREE    ---
1      80        80      ALLOCATED 101
10     160       120     FREE    ---
2      280       50      FREE    ---
3      330       100     FREE    ---
4      430       30      FREE    ---
5      460       120     FREE    ---
6      580       40      FREE    ---
7      620       150     FREE    ---
8      770       60      FREE    ---
9      830       194     FREE    ---
-----
```

```
>> Deallocate PID 101
[FREE] Process 101 block 1 (80 KB) deallocated
[MERGE] Blocks 0 and 1 merged into one 160 KB block
[MERGE] Blocks 0 and 10 merged into one 280 KB block
[MERGE] Blocks 0 and 2 merged into one 330 KB block
[MERGE] Blocks 0 and 3 merged into one 430 KB block
[MERGE] Blocks 0 and 4 merged into one 460 KB block
[MERGE] Blocks 0 and 5 merged into one 580 KB block
[MERGE] Blocks 0 and 6 merged into one 620 KB block

[Worst-Fit] Memory State:
-----
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0         620     FREE    ---
7      620       150     ALLOCATED 104
8      770       60      FREE    ---
9      830       45      ALLOCATED 103
10     875       149     FREE    ---
-----

>> Deallocate PID 103
[FREE] Process 103 block 0 (45 KB) deallocated
[MERGE] Blocks 8 and 9 merged into one 105 KB block
[MERGE] Blocks 8 and 10 merged into one 254 KB block

[Worst-Fit] Memory State:
-----
BlkID  Start(KB)  Size(KB)  Status  PID
-----
0      0         620     FREE    ---
7      620       150     ALLOCATED 104
8      770       254     FREE    ---
-----

--- Fragmentation Report [Worst-Fit] ---
Total Free Memory : 874 KB
Total Allocated : 150 KB
Free Block Count : 2
Largest Free Block : 620 KB
External Frag : YES (scattered free blocks)

===== STRESS TEST [Worst-Fit] =====
Stress req #1: 16 KB (PID 900) -> [SPLIT] Block 0 split: 16 KB allocated, 604 KB returned as Block 9
Stress req #2: 70 KB (PID 901) -> [SPLIT] Block 9 split: 70 KB allocated, 534 KB returned as Block 9
Stress req #3: 87 KB (PID 902) -> [SPLIT] Block 9 split: 87 KB allocated, 447 KB returned as Block 9
Stress req #4: 139 KB (PID 903) -> [SPLIT] Block 9 split: 139 KB allocated, 308 KB returned as Block 9
Stress req #5: 86 KB (PID 904) -> [SPLIT] Block 9 split: 86 KB allocated, 222 KB returned as Block 9
Stress req #6: 77 KB (PID 905) -> [SPLIT] Block 8 split: 77 KB allocated, 177 KB returned as Block 9
Stress req #7: 83 KB (PID 906) -> [SPLIT] Block 9 split: 83 KB allocated, 139 KB returned as Block 10
Stress req #8: 73 KB (PID 907) -> [SPLIT] Block 9 split: 73 KB allocated, 104 KB returned as Block 10
Stress req #9: 15 KB (PID 908) -> [SPLIT] Block 10 split: 15 KB allocated, 124 KB returned as Block 11
Stress req #10: 12 KB (PID 909) -> [SPLIT] Block 11 split: 12 KB allocated, 112 KB returned as Block 11
Result: 10 allocated, 0 failed

--- Fragmentation Report [Worst-Fit] ---
```

10. Conclusion

This lab successfully implemented all four classical dynamic memory allocation algorithms — First-Fit, Next-Fit, Best-Fit, and Worst-Fit — each running against an independent copy of the same 1,024 KB, 10-block memory pool. Block splitting (threshold: 20 KB), block merging/coalescing on deallocation, and a 10-request randomised stress test were all implemented and verified.

Key findings from the simulation and stress test:

- No single algorithm dominates in all metrics. The best choice depends on the workload characteristics and system constraints.
- Coalescing is essential for all algorithms — without it, external fragmentation grows unbounded regardless of allocation strategy.
- Best-Fit provides the best memory utilisation per allocation but accumulates the most small holes over time.
- Worst-Fit performed best in the stress test for this specific workload (10/10 success) due to a large free block being available after coalescing — but this advantage disappears once that block is consumed.
- First-Fit and Next-Fit offer the best balance of speed and simplicity for general workloads.